

Facultad de Ingeniería, Universidad de Buenos Aires

Sistemas Distribuidos

TP diseño

Grupo 15

Integrante	**Padrón**
Basso, Catalina	108564
Favotti, Franco	109388
Fernandez Delgado, Victoria	110545

24 de octubre de 2026

Índice

1. Introducción **2**
1.1. Descripción del sistema 2
1.2. Consultas a resolver 2

2. Vista de Arquitectura **2**

3. Escenarios **3**
3.1. Casos de Uso 3

4. Vista Lógica **4**
4.1. DAG 4

5. Vista de Procesos **7**
5.1. Diagrama de actividades 7
5.2. Diagrama de secuencia 9

6. Vista de Desarrollo **14**
6.1. Diagrama de Paquetes 14

7. Vista Física **15**
7.1. Diagrama de Robustez 15
7.2. Diagrama de Despliegue 20

8. Tareas y división entre integrantes **22**

1 Introducción

En este trabajo práctico, se diseña un sistema para analizar un conjunto de artículos de Wikipedia en Ingles y en Francés. Se utiliza el modelo de vista 4+1 para presentarlo.

1.1 Descripción del sistema

El sistema recibe un conjunto de artículos desde uno o mas clientes, los procesa de forma distribuida mediante un pipeline de etapas conectadas por cola y devuelve a cada cliente los resultados de las cinco consultas, a medida que se van produciendo.

1.2 Consultas a resolver

El análisis se realiza mediante consultas, que se resuelven de distintas maneras.

1. Url y fecha de modificación para artículos escritos en inglés hasta 2020 inclusive.
2. Nombre, url e imágen (bytes) de artículos escritos hasta 2020 inclusive que incluyan "biography" o "biographie" entre sus secciones.
3. Listado de palabras únicas compartidas entre cualquier artículo escrito en inglés y cualquier artículo escrito en francés.
4. Pares (palabra, cantidad) del top 20 palabras más utilizadas entre todos los abstracts. Dentro de un mismo abstract las palabras se cuentan una sola vez y las palabras triviales (stopwords) se excluyen del top.
5. Url y cantidad de palabras de abstract para los artículos escritos a partir del 2021 cuya cantidad de palabras en el abstract sea superior al promedio de palabras de abstract de todos los artículos escritos hasta el 2020 inclusive. Se aplica el mismo criterio para contabilizar palabras que la query anterior.

2 Vista de Arquitectura

El cliente se conecta mediante TCP a un Gateway. El Gateway recibe los batches de artículos, les asigna el `client_id` y los publica en la cola `year_in`, de la que consumen las réplicas del `YearFilter`. Este filtro es el único que interpreta el año: envía a `q1_in` y `q2_in` solo los artículos hasta 2020 (Q1 y Q2) y a `norm_in` todos los artículos, marcados con su período (hasta 2020 o desde 2021), para el procesamiento de texto de Q3, Q4 y Q5. En cada etapa hay réplicas que consumen en paralelo de colas del middleware (RabbitMQ). Las etapas particionadas (`WordReducer`, `AvgReducer` y `Q5Filter`) tienen una cola por partición con un único consumidor. La única etapa de agregación global es el `TopMerger`, necesaria porque los datos de un cliente para Q4 quedan repartidos entre las particiones de palabras. Los resultados de las cinco consultas viajan a un `Joiner` por Gateway, que los reenvía al cliente a través de su Gateway.

Componente	Responsabilidad	Instancias
Client	Arma los batches, los envía por TCP y recibe los resultados, que escribe en un archivo por consulta.	C
Gateway	Acepta clientes, asigna a cada sesión un `client_id`= (`gw_id`, n), publica batches y EOF en `year_in` y devuelve a su cliente los resultados que le entrega su Joiner. Incluye las funciones del antiguo `MessageHandler`.	G
YearFilter	Interpreta la fecha de modificación. Envía a Q1 y Q2 los artículos hasta 2020 y a la rama de texto todos los artículos con su período.	N
Q1Filter	Filtra artículos en inglés (Q1).	N
Q2Filter	Filtra artículos con sección "biography"/"biographie" (Q2).	N
ImageGetter	Descarga la imagen del artículo desde la API de Wikimedia.	N
AbstractCleaner	Limpia el abstract: palabras únicas, sin stopwords ni números. Las stopwords son una constante del código.	N
WordMapper	Arma los mapas parciales de palabras y los envía a la partición que corresponde a cada palabra.	N
LengthMapper	Calcula n (cantidad de palabras únicas del abstract). Para artículos hasta 2020 suma n y la cantidad por batch y la envía al promedio; para los de 2021 en adelante envía {`client_id`, `url`, n} a Q5.	N
WordReducer	Combina los conteos de las palabras de su partición: resuelve Q3 y aporta el top local de Q4.	P
TopMerger	Calcula el top 20 global a partir de los tops locales (Q4).	1
AvgReducer	Acumula Σn y cantidad por cliente; al EOF calcula el promedio del cliente y lo publica. Particionado por `client_id`.	A
Q5Filter	Guarda en un spool local los artículos desde 2021 y, con el promedio del cliente, emite los que lo superan (Q5). Particionado por url.	K
Joiner	Reenvía los resultados de los clientes de su Gateway y cuenta los EOF por consulta (1 para Q1, Q2 y Q4, P para Q3 y K para Q5).	G

Cuadro 1: Componentes del sistema. C clientes, G gateways, N réplicas que compiten por una cola, P particiones de palabras, A particiones del promedio y K particiones de Q5. P, A y K se fijan al desplegar.

3 Escenarios

3.1 Casos de Uso

- El cliente envia el dataset y recibe los resultados de todas las consultas.
- Se ejecutan varios clientes a la vez, repartidos entre varios Gateways.
- Al recibir un SIGTERM, cada proceso finaliza su trabajo, liberando recursos.

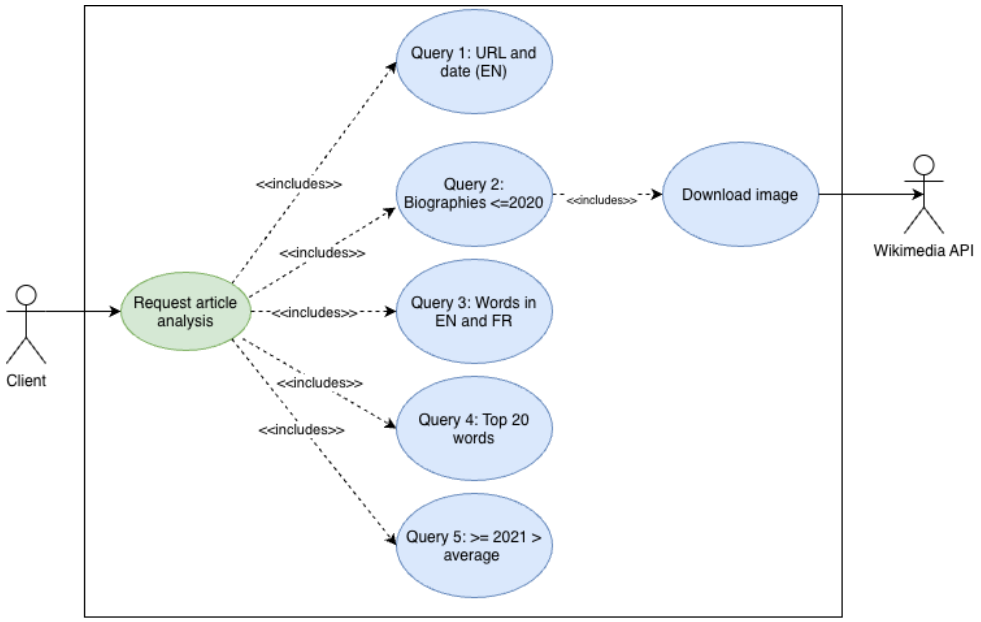


Figura 1: Diagrama de casos de uso

4 Vista Lógica

4.1 DAG

La lectura de artículos se divide en dos ramas, una donde están los filtros de las primeras dos consultas y otra donde se procesa el texto del resto. Ambas parten del `YearFilter`.

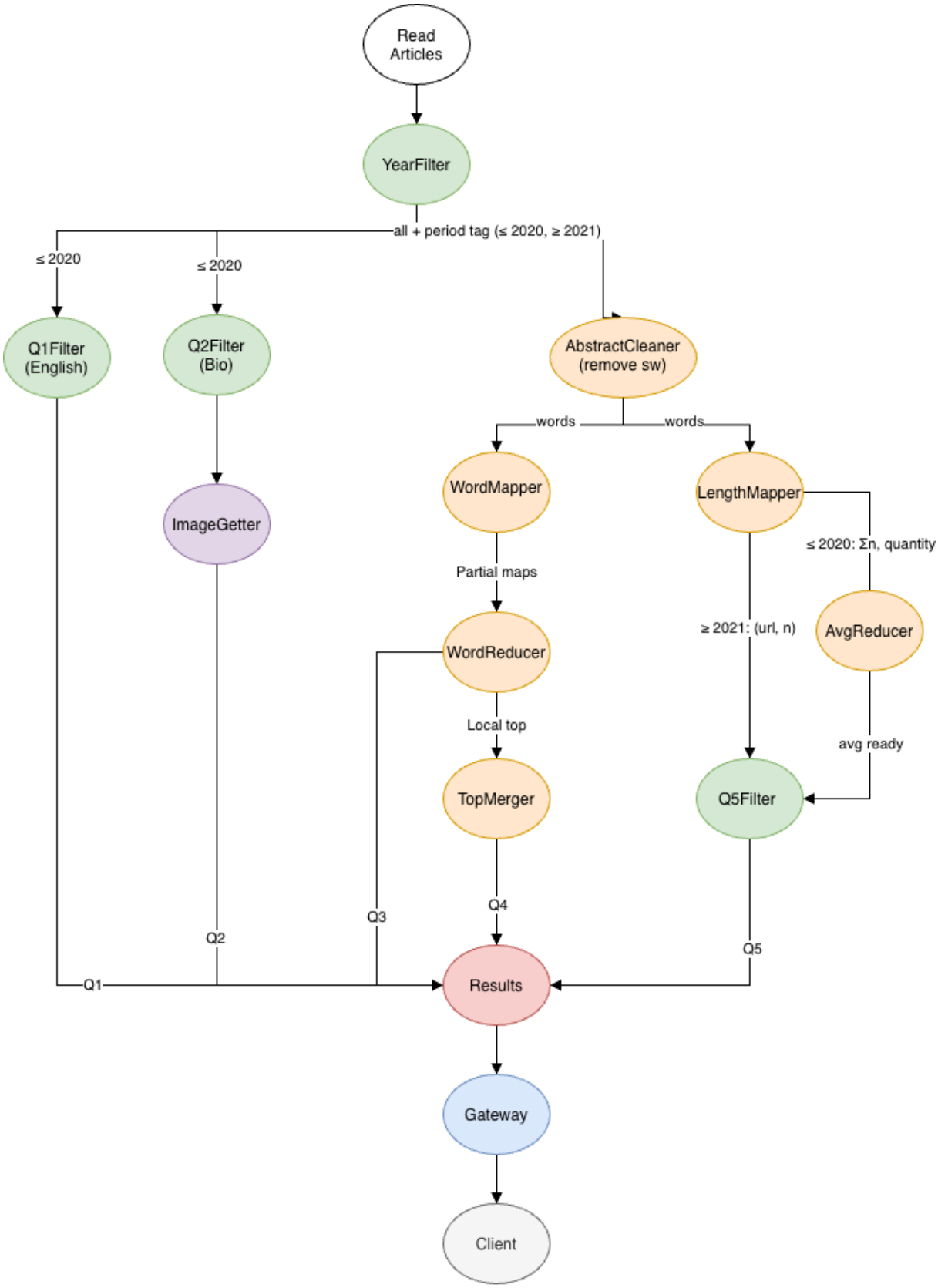


Figura 2: DAG

En la primera rama, `Q1Filter` deja los artículos en inglés (Q1) y `Q2Filter` los que incluyen una sección de biografía, que pasan por `ImageGetter` para obtener la imagen (Q2); ambos reciben solo artículos hasta 2020 porque el `YearFilter` ya descartó el resto. En la segunda

rama, el `AbstractCleaner` limpia el abstract y el resultado se bifurca: el `WordMapper` reparte las palabras entre los `WordReducer`, que resuelven Q3 y alimentan al `TopMerger` (Q4), y el `LengthMapper` calcula la cantidad de palabras n de cada artículo. Para los artículos hasta 2020 el `LengthMapper` alimenta a los `AvgReducer`, particionados por cliente, que calculan el promedio de cada cliente. Para los artículos desde 2021 los envía a los `Q5Filter`, particionados por url, que los comparan con el promedio de su cliente (Q5). Los resultados convergen en el `Joiner` del Gateway del cliente, que los entrega a ese Gateway y este al cliente. Hay un `TopMerger` y no un agregador de promedio: un agregador global solo hace falta cuando los datos de un cliente quedan repartidos entre nodos. Eso pasa con Q4: las palabras de un cliente se reparten entre las P particiones, y el top 20 global exige unir los P tops locales. En el promedio, en cambio, la partición es por `client_id`: cada cliente cae entero en un único `AvgReducer`, que ya puede calcular su promedio sin ningún nodo posterior.

Q5 se particiona por url y no por cliente porque compara cada artículo solo contra el promedio de su cliente, por lo que no necesita ver juntos los artículos de un mismo cliente. Si se particionara por cliente, un cliente grande cargaría un único nodo y un único disco, y aumentar K no lo aliviaría. Al particionar por url, los artículos de un cliente se reparten entre las K particiones, cada url se evalúa una sola vez y el costo es que el promedio se envía a las K réplicas (mensajes muy chicos).

A continuación se incluye el recorrido de cada consulta por separado.

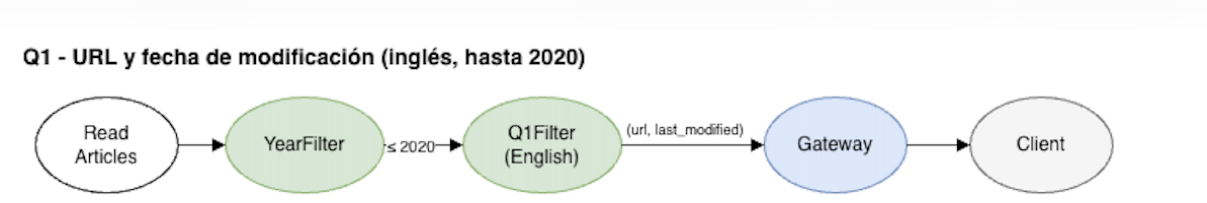


Figura 3: DAG de Q1.

El `YearFilter` descarta los artículos posteriores a 2020 y `Q1Filter` se queda con los escritos en inglés. Es una etapa sin estado que emite `(url, last_modified)` hacia el Gateway.

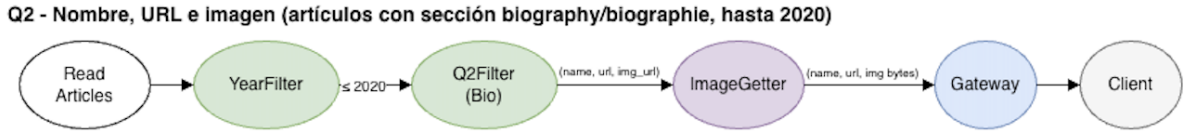


Figura 4: DAG de Q2.

`Q2Filter` conserva los artículos hasta 2020 con una sección “biography” o “biographie” y se los pasa al `ImageGetter`, que descarga la imagen y emite `(name, url, img bytes)`. La descarga es la parte lenta del recorrido, por eso esta etapa se replica por separado.



Figura 5: DAG de Q3.

Todos los artículos, sin filtro de año, pasan por el `AbstractCleaner`. El `WordMapper` emite `(word, lang)` y el `WordReducer` registra, para cada palabra, si apareció en inglés y/o en francés. Las palabras que completan ambos idiomas son el resultado de Q3.



Figura 6: DAG de Q4.

Comparte las etapas de Q3 hasta el `WordReducer`, que además cuenta en cuántos abstracts aparece cada palabra (una vez por abstract, porque el `AbstractCleaner` ya dejó palabras únicas). Cada reducer envía su top 20 local al `TopMerger`, que los une en el top 20 global.



Figura 7: DAG de Q5.

Tras la limpieza, el `LengthMapper` calcula la cantidad de palabras n de cada abstract. Los artículos hasta 2020 aportan Σn y cantidad al `AvgReducer`, que calcula el promedio. Los artículos desde 2021 van como `(url, n)` al `Q5Filter`, que los compara contra el promedio cuando este llega ("avg ready").

5 Vista de Procesos

5.1 Diagrama de actividades

Muestra el flujo desde que el cliente arma los batches hasta que recibe los resultados. El Gateway asigna el `client_id` y publica cada batch; el `YearFilter` lo clasifica, y desde ahí se ve la división en las dos ramas y la unión en el Joiner. Los resultados parciales viajan por las flechas. `Q3` y `Q4` se muestran separados porque salen de componentes distintos: cada `WordReducer` entrega al Joiner las palabras compartidas de su partición y envía su top 20 local al `TopMerger`, que une los tops y entrega Q4. `Q5Filter` guarda los artículos desde 2021 en el spool y recién los compara cuando recibe el promedio del cliente. Los resultados se envían al cliente a medida que se producen; el Joiner lleva la cuenta de EOF por consulta (en Q3 uno por cada partición de WordReducer y en Q5 uno por cada partición de Q5Filter) y, al completarlos, avisa que el cliente terminó.

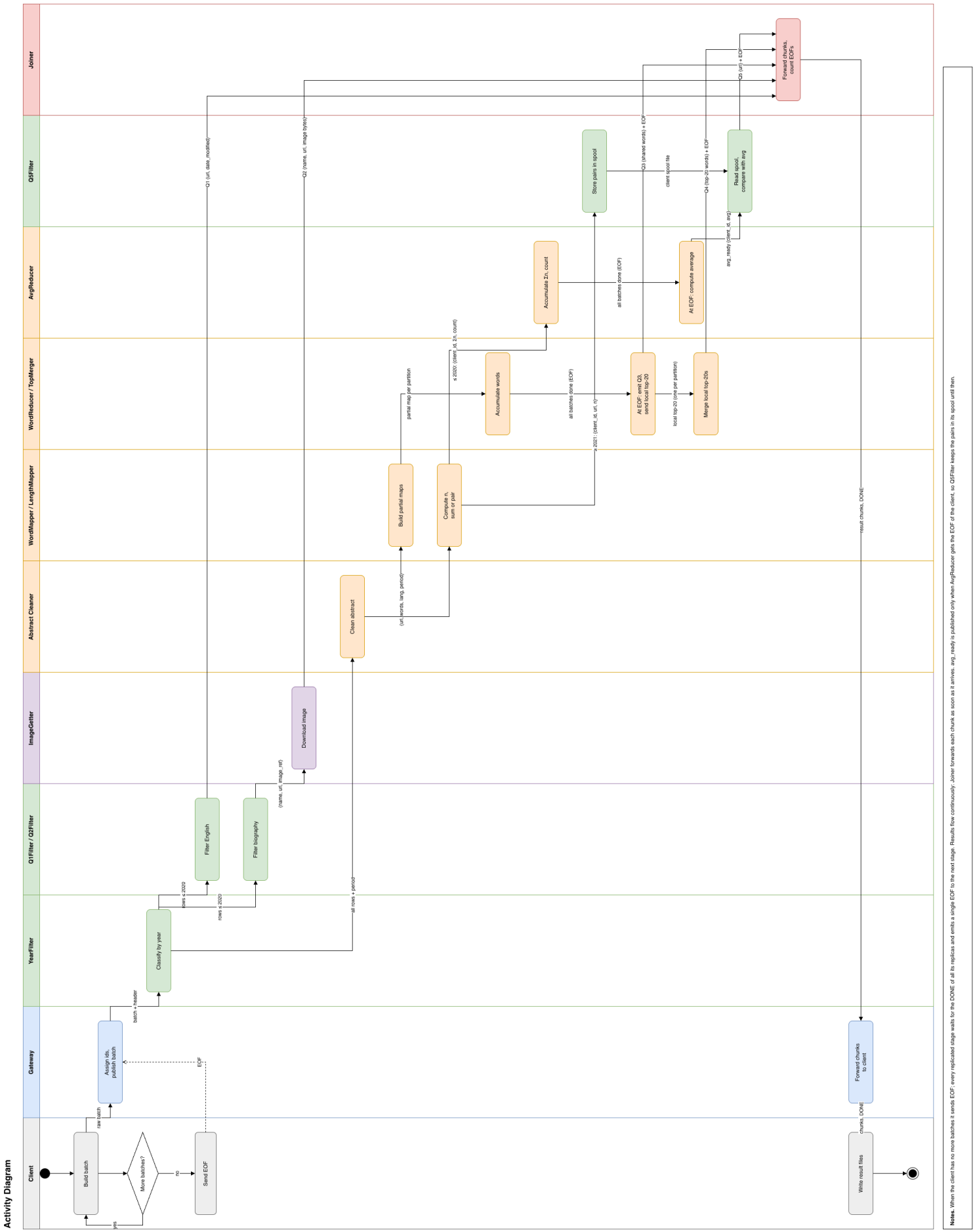


Figura 8: Diagrama de Actividades

5.2 Diagrama de secuencia

Los diagramas de secuencia muestran, para cada consulta, cómo se maneja la información en cada etapa y cuándo está lista cada respuesta. Todos se dividen en los mismos momentos:

1. El procesamiento de cada batch (bloque `loop`).
2. La propagación del EOF (bloque `EOF`): cada etapa espera el EOF de todas las réplicas anteriores antes de enviar el suyo.
3. El cálculo de los resultados que dependen de haber visto todos los datos del cliente, en un bloque propio: el resultado final de Q3, el top 20 global de Q4 y la barrera del promedio de Q5.

Para mantener los diagramas legibles se omite el `Joiner`: los resultados se dibujan llegando directamente al Gateway. En el sistema real pasan por `results_ex` y por el `Joiner` del Gateway del cliente, que los reenvía a medida que llegan y cuenta los EOF por consulta y por cliente: uno de Q1, Q2 y Q4, P de Q3 (uno por partición de `WordReducer`, identificadas por el campo `origin`) y K de Q5 (uno por cada `Q5Filter`). Como cada consulta se muestra por separado, no se ve que las ramas de Q3/Q4 y de Q5 se ejecutan en paralelo en el sistema.

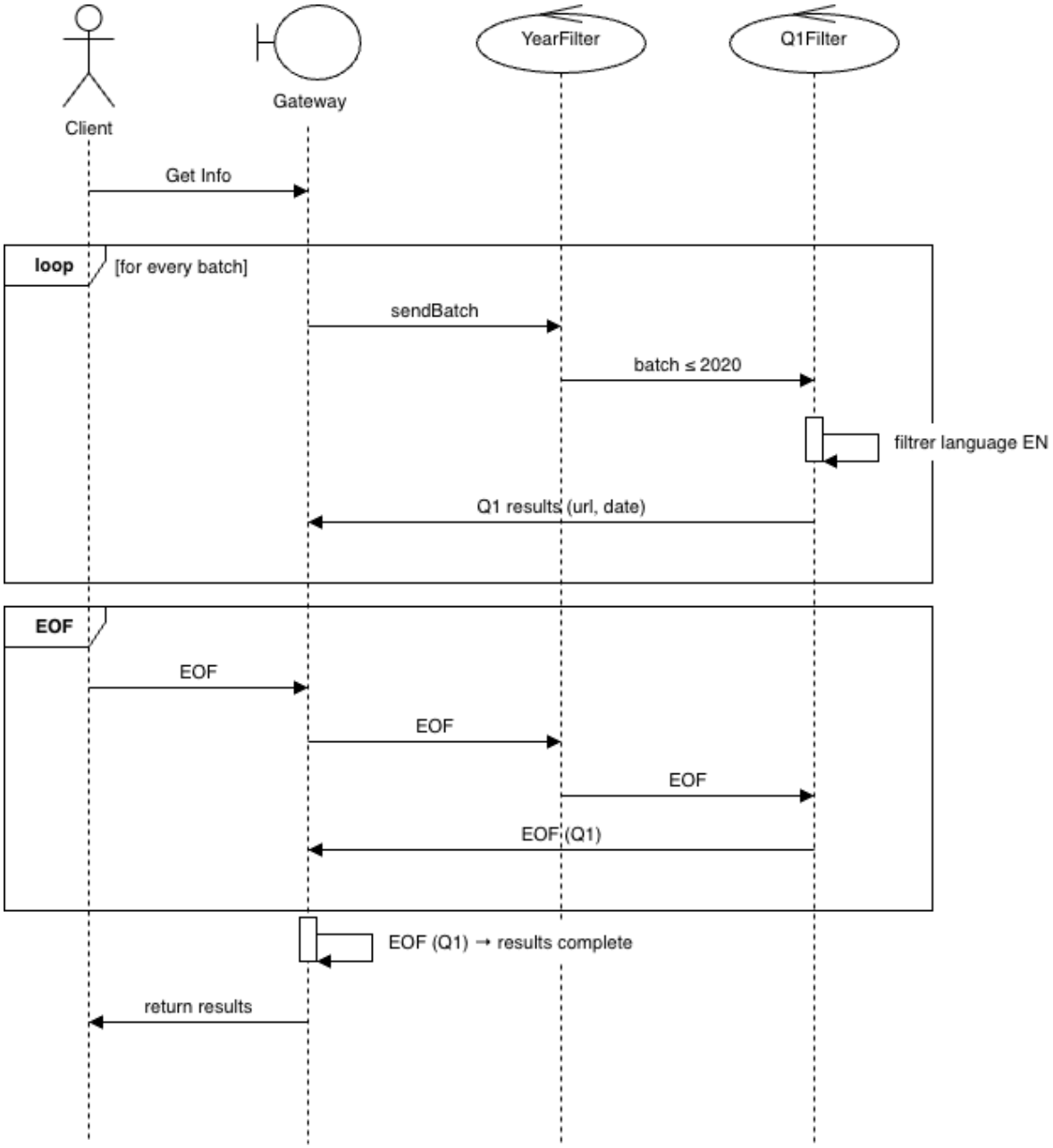


Figura 9: Diagrama de secuencia de Q1.

Por cada batch, el Gateway lo publica, el `YearFilter` lo reenvía a `Q1Filter` solo con los artículos hasta 2020 y este filtra por idioma inglés y devuelve `(url, date)`. El EOF del cliente se propaga por el Gateway, el `YearFilter` y el `Q1Filter`; cuando vuelve el EOF de Q1, el Gateway da por completos los resultados y los devuelve al cliente.

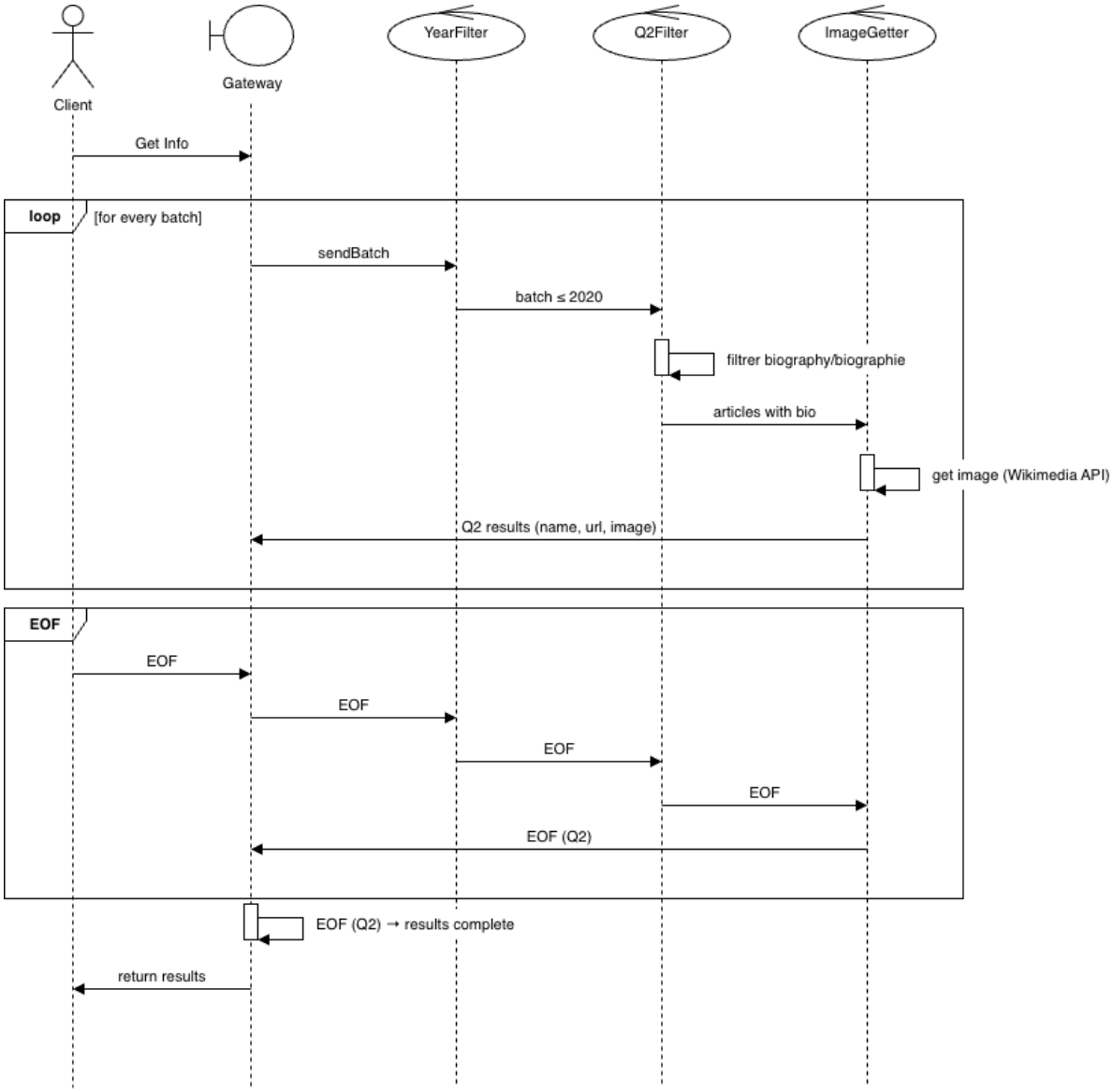


Figura 10: Diagrama de secuencia de Q2.

Igual que Q1, pero `Q2Filter` conserva los artículos con sección de biografía y los pasa al `ImageGetter`, que descarga la imagen desde la API de Wikimedia y devuelve `(name, url, image)` al Gateway. El EOF recorre la misma cadena de etapas.

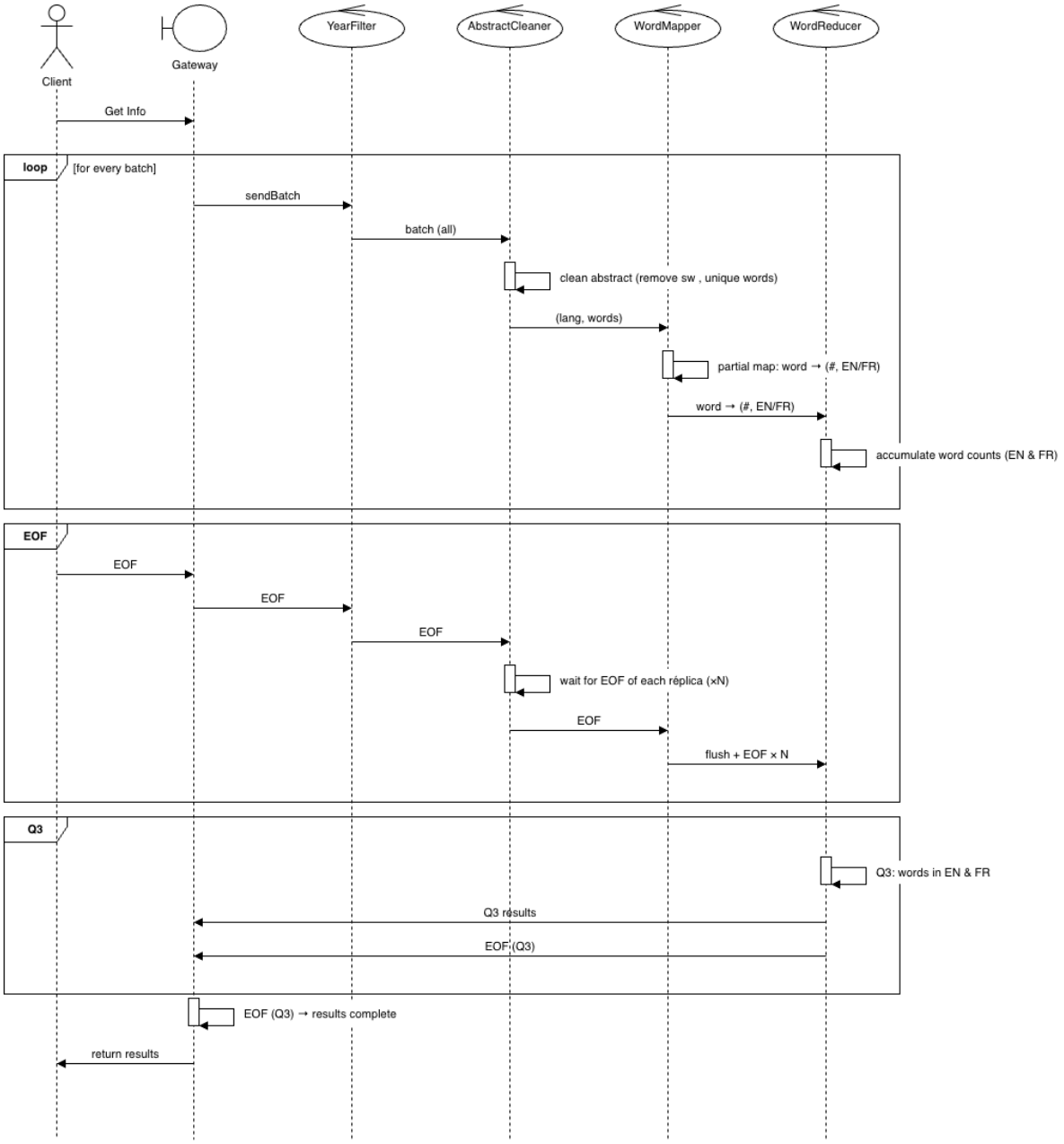


Figura 11: Diagrama de secuencia de Q3.

En el `loop`, el `AbstractCleaner` limpia el abstract, el `WordMapper` arma el mapa parcial `word` $\rightarrow$ (#, EN/FR) y el `WordReducer` lo acumula. En el bloque `EOF`, cada etapa espera el EOF de todas las réplicas anteriores y el `WordMapper` vuelca sus parciales junto con su EOF. Recién entonces, en el bloque `Q3`, el `WordReducer` determina las palabras presentes en inglés y francés y las envía con su EOF.

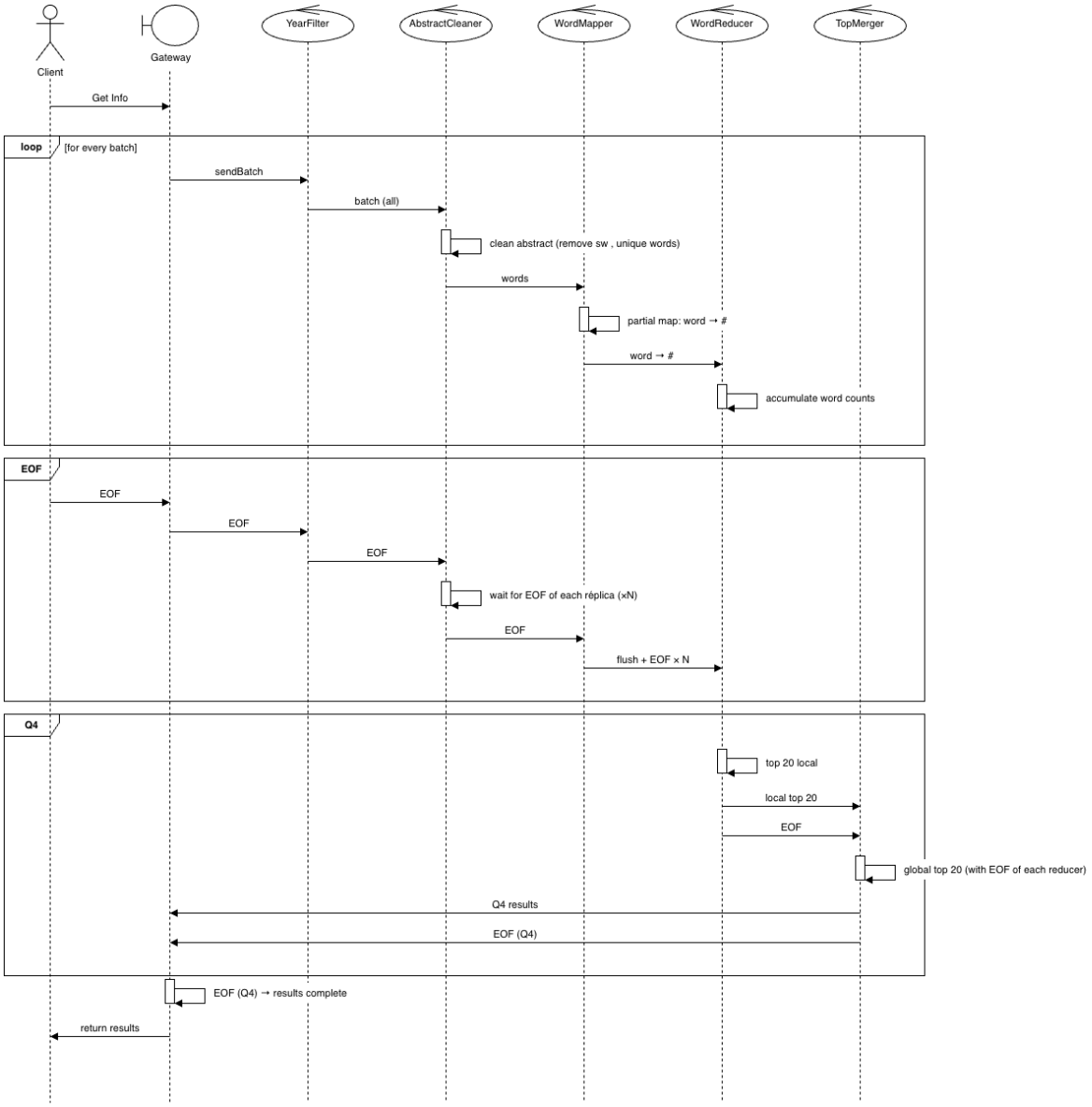


Figura 12: Diagrama de secuencia de Q4.

El procesamiento por batch y el EOF son iguales a los de Q3. En el bloque `Q4`, cada `WordReducer` calcula su top 20 local y lo envía al `TopMerger` con su EOF. El `TopMerger` espera el EOF de todos los reducers y recién entonces calcula y envía el top 20 global.

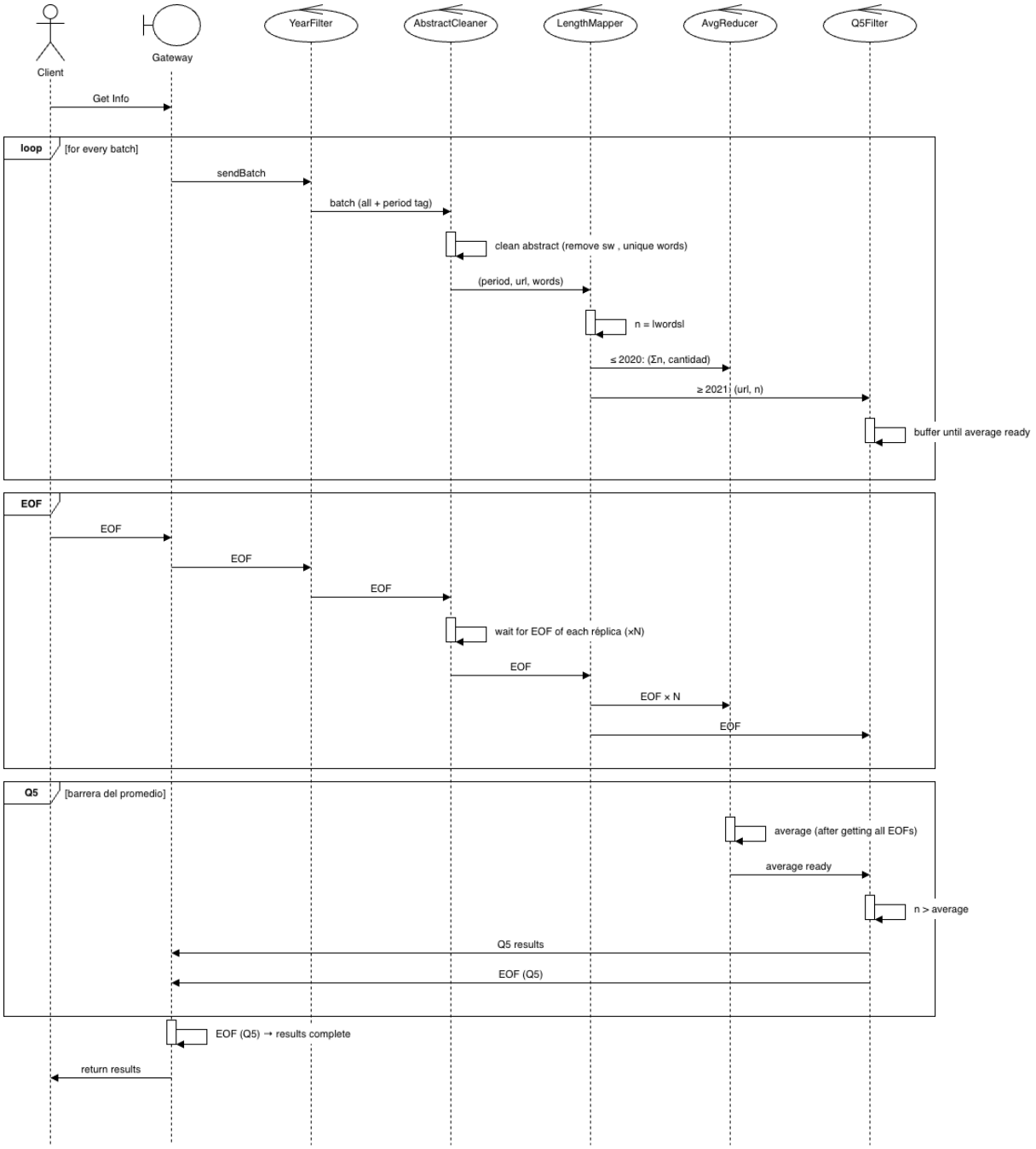


Figura 13: Diagrama de secuencia de Q5.

En el `loop`, el `LengthMapper` calcula n: los artículos hasta 2020 se acumulan como $(\Sigma n, \text{cantidad})$ hacia el `AvgReducer` y los de 2021 en adelante van como `(url, n)` al `Q5Filter`, que los guarda en su buffer local. En el bloque `Q5` (barrera del promedio), el `AvgReducer` calcula el promedio solo después de recibir todos los EOF y lo difunde al `Q5Filter`, que lee su buffer y emite los artículos con n mayor al promedio.

6 Vista de Desarrollo

6.1 Diagrama de Paquetes

El código se organiza en paquetes por responsabilidad:

- `gateway` contiene el servidor TCP y las funciones que antes estaban en el `MessageHandler`.
- `filters` los filtros `YearFilter`, `Q1Filter`, `Q2Filter` y `Q5Filter`.
- `text_processing` el `AbstractCleaner`, el `WordMapper`, el `LengthMapper`, el `WordReducer`, el `TopMerger` y el `AvgReducer`.
- `image` el ImageGetter, que es el único que consume la API externa de Wikimedia por HTTPS.
- `results` el Joiner.
- `client` se comunica con el Gateway por TCP y escribe los resultados.

Todos los paquetes dependen de `common`, que agrupa lo compartido: `middleware` (productores y consumidores, colas de trabajo, exchanges y manejo de EOF), `protocol` (batches, artículos, resultados y serialización de mensajes) y `utils` (graceful shutdown ante SIGTERM, configuración, logging, la constante `STOPWORDS` y el `Spool` de archivos locales). El `Spool` vive en `utils` porque solo sabe escribir, leer y borrar archivos por cliente, sin conocer las reglas de Q5: `Q5Filter` lo usa como dependencia. Los paquetes de negocio no dependen entre sí: `gateway` y `results`, por ejemplo, se comunican únicamente por las colas del middleware.

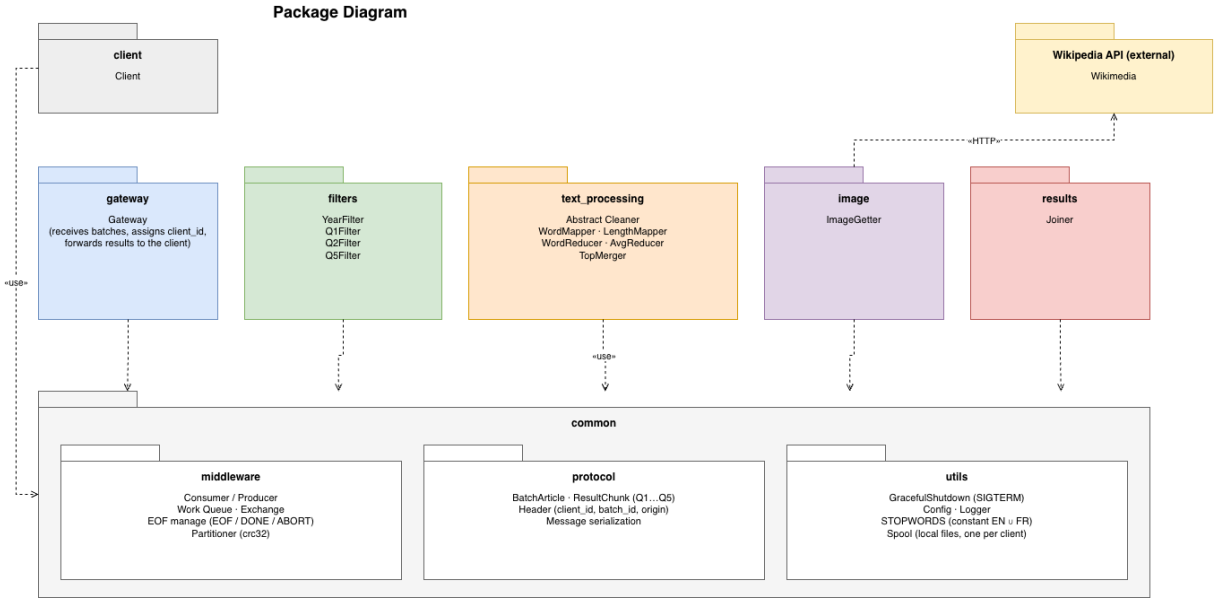


Figura 14: Diagrama de paquetes

7 Vista Física

7.1 Diagrama de Robustez

El diagrama muestra los procesos del sistema, las colas y exchanges que los conectan y los datos que intercambian.

- **Entrada:** Cada Gateway recibe los batches de sus clientes, les asigna el `client_id`= (`gw_id`, n) y los publica en `year_in`, de la que compiten las réplicas de `YearFilter`. Cada mensaje lleva un encabezado común con `client_id`, `batch_id` y origen.
- **Clasificación por año:** El `YearFilter` es el único que calcula el año. Publica en `q1_in` y `q2_in` solo los artículos hasta 2020 (con los campos que necesita cada consulta) y en `norm_in` todos los artículos con su período.

- **Q1 y Q2:** `Q1Filter` y `Q2Filter` aplican el filtro por idioma o sección. `Q2Filter` envía solo una referencia de imagen a `img_in`; `ImageGetter` descarga los bytes desde la API de Wikimedia.
- **Procesamiento de texto:** El `AbstractCleaner` limpia cada abstract (palabras únicas, sin stopwords ni números; la lista de stopwords es una constante del código, sin lectura de archivos) y publica el resultado en `tok_ex`, un exchange fanout que lo entrega al `WordMapper` y al `LengthMapper`.
- **Q3 y Q4:** El `WordMapper` envía mapas parciales de palabras a `words_ex`, un exchange direct que particiona por `crc32(palabra) % P` hacia las colas `words_q[i]`, una por partición con un único consumidor. Una misma palabra siempre llega al mismo `WordReducer[i]`. Cada reducer envía directamente a `results_ex` los resultados de Q3 de su partición con su EOF, y su top local a `top_in`; el `TopMerger` espera el EOF de las P particiones y obtiene el top 20 global (Q4).
- **Promedio:** El `LengthMapper` calcula n para cada artículo. Para los artículos hasta 2020 envía por batch $\{$`client_id`$, \Sigma n,$ `cantidad`$\}$ a `avg_pex`, que particiona por `crc32(client_id) % A` hacia `avg_in[a]`. Cada `AvgReducer[a]` recibe clientes completos, acumula Σn y cantidad por cliente y, al EOF, calcula el promedio y lo publica como `avg_ready`$\{$`client_id`$,$ `avg`$\}$ en `avg_ex`, un exchange fanout hacia las K réplicas de `Q5Filter`. No hay nodo posterior: cada cliente está completo en una partición.
- **Q5:** Para los artículos desde 2021 el `LengthMapper` envía $\{$`client_id`$,$ `url`$, n\}$ a `q5_pex`, que particiona por `crc32(url) % K` hacia `q5_in[k]`. Cada `Q5Filter[k]` recibe pares de muchos clientes mezclados: los agrega a un archivo en el disco local (spool) por cliente y confirma el mensaje, por lo que el broker no acumula los artículos. Al recibir el EOF de su cola y el `avg_ready` de un cliente, lee el archivo de ese cliente, emite a `results_ex` las urls con $n >$ `avg` junto con su EOF y borra el archivo. Cada cliente tiene así como máximo un archivo por partición.
- **Control:** Los exchanges `*_ctrl` permiten que las réplicas de una misma etapa se coordinen para el EOF antes de propagarlo. Un exchange `abort_ex` propaga `abort`$\{$`client_id`$\}$ para liberar el estado de un cliente que se desconecta.
- **Salida:** Todos los productores de resultados (Q1Filter, ImageGetter, WordReducer, TopMerger y Q5Filter) publican en `results_ex`, un exchange direct con routing key `gw_id`, que los deriva a `results_q[g]`. `Joiner[g]` reenvía cada fragmento a `out_q[g]` y cuenta los EOF por consulta y por `client_id` $(1 + 1 + P + 1 + K)$. El Gateway usa la parte n del `client_id` para entregar cada fragmento a su cliente por la conexión correspondiente. No hay volumen compartido: el cliente escribe un archivo por consulta.

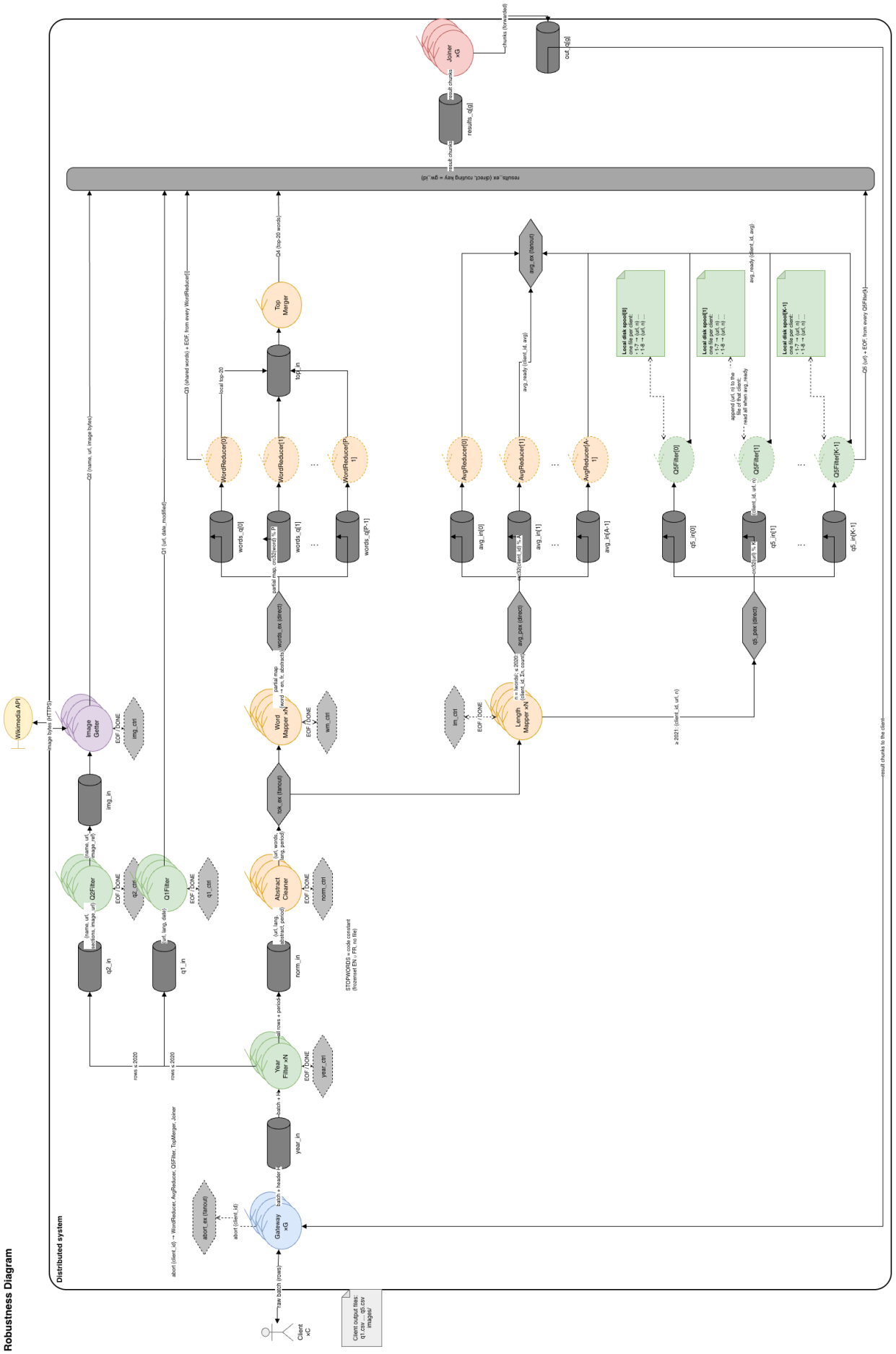


Figura 15: Diagrama de robustez

A continuación se detallan los diagramas de robustez por Query:

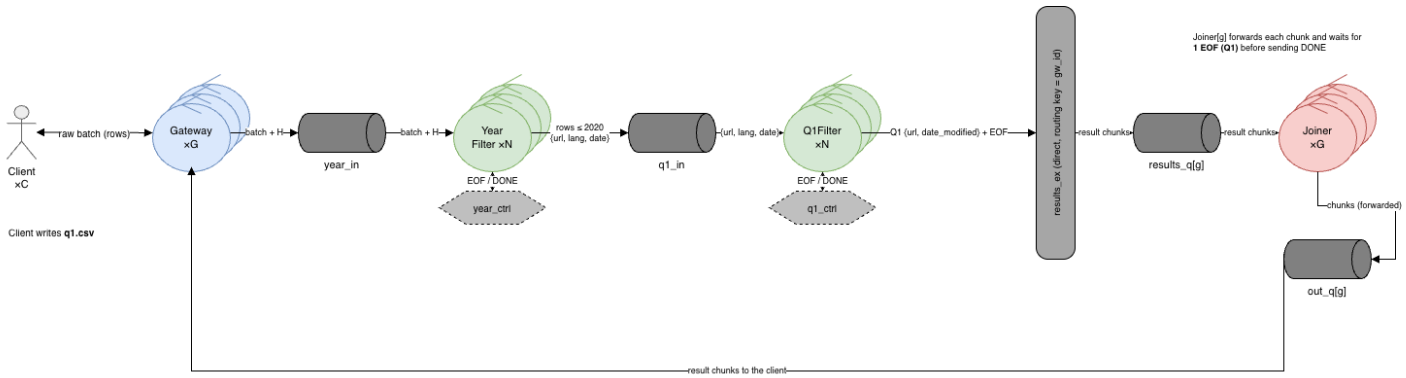


Figura 16: Diagrama de robustez Q1

El Gateway publica los batches en `year_in`; el `YearFilter` deja en `q1_in` solo los artículos hasta 2020 y `Q1Filter` los filtra por idioma. Publica `(url, date_modified)` y su EOF en `results_ex`, que los deriva a `results_q[g]`. El `Joiner[g]` reenvía cada fragmento a `out_q[g]` y espera un solo EOF antes de avisar que Q1 terminó. El cliente escribe `q1.csv`.

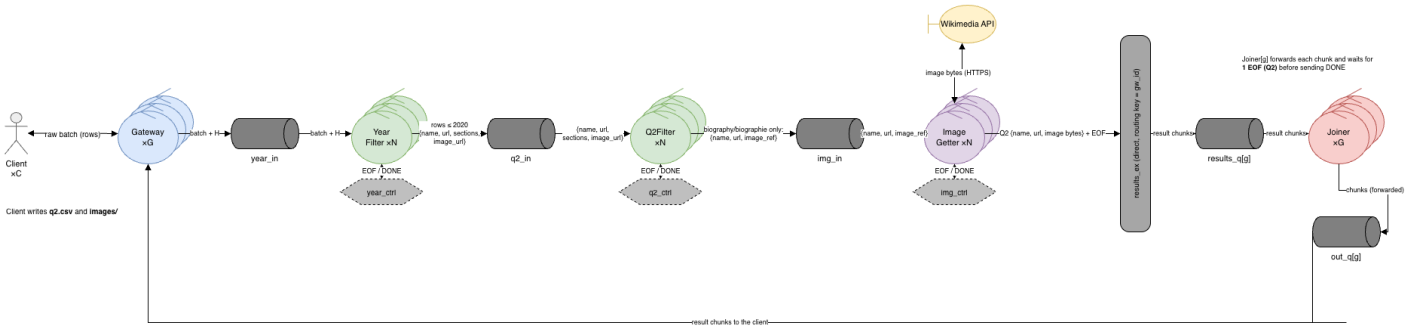


Figura 17: Diagrama de robustez Q2

El `YearFilter` deja en `q2_in` los artículos hasta 2020 con los campos que necesita la consulta. `Q2Filter` conserva los de sección de biografía y envía solo una referencia de imagen a `img_in`, para no mover bytes por la cola. `ImageGetter` descarga la imagen por HTTPS desde la API de Wikimedia y publica `(name, url, image bytes)` con su EOF en `results_ex`. El `Joiner` espera un EOF y el cliente escribe `q2.csv` y la carpeta `images/`.

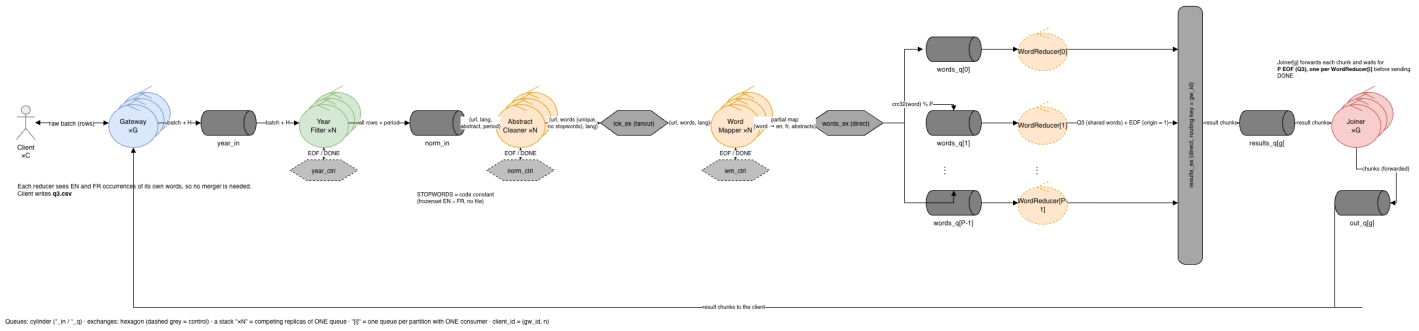


Figura 18: Diagrama de robustez Q3

El `AbstractCleaner` publica en `tok_ex` y el `WordMapper` envía sus mapas parciales a `words_ex`, que particiona por `crc32(palabra) % P`. Una misma palabra siempre llega al mismo `WordReducer[i]`, que ve todos sus idiomas y decide por sí solo si es compartida, por lo que no hace falta un nodo posterior. Cada reducer envía a `results_ex` las palabras compartidas de su partición con su EOF, y el `Joiner` espera P EOF.

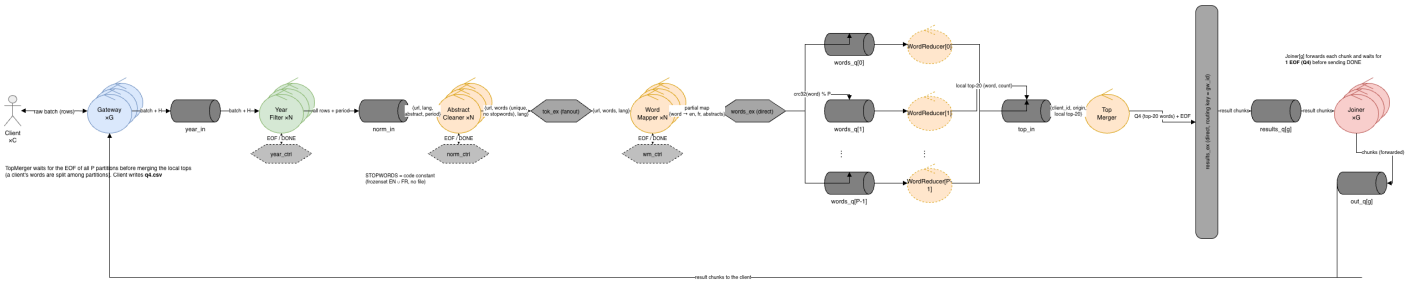


Figura 19: Diagrama de robustez Q4

El recorrido es el de Q3 hasta el `WordReducer[i]`, que además cuenta las palabras de su partición y envía su top 20 local a `top_in`. El `TopMerger` espera el EOF de las P particiones y obtiene el top 20 global, que publica en `results_ex`. Es la única etapa de agregación global, necesaria porque las palabras de un cliente quedan repartidas entre particiones. El `Joiner` espera un solo EOF.

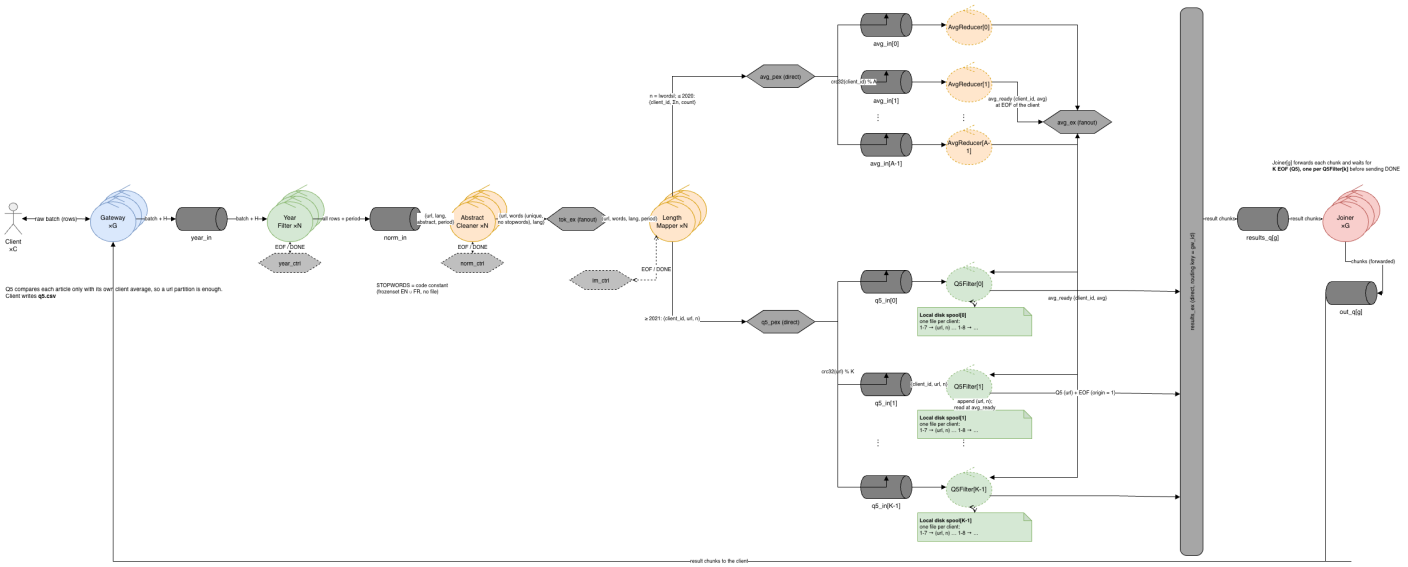


Figura 20: Diagrama de robustez Q5

El `LengthMapper` separa dos flujos. Para los artículos hasta 2020 envía por batch $\{$`client_id`, Σn, cantidad$\}$ a `avg_pex`, que particiona por `crc32(client_id) % A`; cada `AvgReducer[a]` recibe clientes completos y, al EOF, publica el promedio en `avg_ex` (fanout a las K réplicas). Para los artículos desde 2021 envía `(client_id, url, n)` a `q5_pex`, que particiona por `crc32(url) % K`. Cada `Q5Filter[k]` agrega los pares a un spool en disco por cliente y confirma el mensaje; al tener el EOF y el promedio del cliente, emite las urls con n mayor al promedio con su EOF y borra el archivo. El `Joiner` espera K EOF.

7.2 Diagrama de Despliegue

El sistema se despliega en nodos independientes, conectados a través del nodo de middleware (RabbitMQ). Los nodos de procesamiento no se comunican directamente entre sí, por lo que pueden replicarse sin modificar al resto. Los nodos particionados (P, A y K) tienen una cola por partición y exactamente un consumidor por cola.

Nodo	**Componentes**	**Réplicas**
Client PC	Client	varias
Gateway Node	Gateway	G
Middleware Node	RabbitMQ	1
Filter Node	YearFilter, Q1Filter, Q2Filter	varias
Image Node	ImageGetter	varias
AbstractCleaner Node	AbstractCleaner	varias
Mapper Node	WordMapper, LengthMapper	varias
Reducer Node	WordReducer	P
Average Node	AvgReducer	A
Q5 Node	Q5Filter y spool en disco local	K
Aggregator Node	TopMerger	1
Joiner Node	Joiner	G

Cuadro 2: Nodos del despliegue.

El cliente solo se conecta con el Gateway, y solo el `Image Node` accede a un servicio externo (API de Wikimedia). El único almacenamiento en disco es el spool local de cada `Q5Filter`, temporal y por cliente; no hay volumen compartido entre nodos. La tolerancia a fallas (persistencia de estado) queda para una etapa posterior.

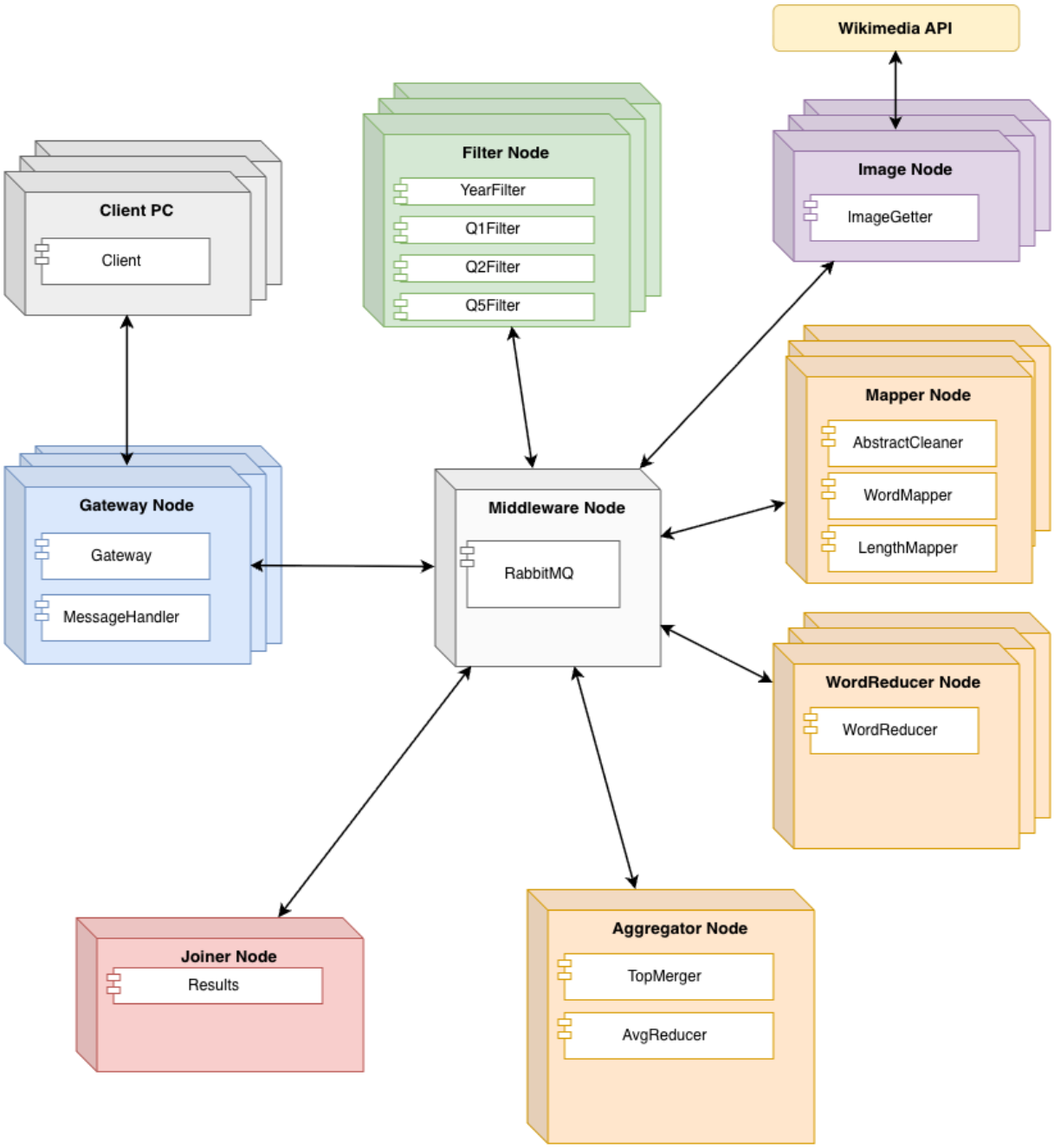


Figura 21: Diagrama de despliegue

8 Tareas y división entre integrantes

N.º	Tarea	Responsable
1	Middleware.	*Catalina Basso*
2	Protocolo.	*Catalina Basso*
3	Utils: configuración, logging, graceful shutdown (SIGTERM), stopwords y spool.	*Catalina Basso*
4	Client y Gateway.	*Franco Favotti*
5	YearFilter, Q1Filter, Q2Filter e ImageGetter (Q1 y Q2).	*Victoria Fernandez Delgado*
6	AbstractCleaner, WordMapper, WordReducer y TopMerger (Q3 y Q4).	*Victoria Fernandez Delgado*
7	LengthMapper, AvgReducer y Q5Filter con spool (Q5).	*Franco Favotti*
8	Joiner por Gateway y entrega de resultados al cliente (conteo de EOF por consulta).	*Franco Favotti*
9	Despliegue.	*Victoria Fernandez Delgado*
10	Tests	Todos